Monotone Classification with Relative Approximations

Lecture Notes for SC2320 Data Analytics and Mining

April 7, 2026

Instructions to readers

The flow of this set of notes is in tandem to the flow of the original paper. It will also go in the same flow as the presentation slides. Concepts/Material/Information that is auxiliary to the core ideas of this paper will be outlined at the end of each section.

Acronym Reference

Acronym	**Full Form**
ER	Entity Resoluton
EM	Entity Matching
ACR3	Acronym Three

1 Background and Motivation

1.1 The Challenge of Entity Matching

Definition 1.1 (Entity Matching)**.** Given two sets of entities E_1 and E_2, the task is to determine, for every pair $(e_1, e_2) \in E_1 \times E_2$, whether e_1 and e_2 refer to the same real-world entity. It is a binary classification task that determines whether a candidate pair is a match $(+1)$ or non-match (-1) [2].

To motivate the problem, consider the following scenario where we have two sets of `entities` that we refer to here as product listings - E_1 and E_2, sourced from two different e-commerce platforms — say Shopee and Lazada respectively. Each listing carries attributes such as `p-name`, `p-description`, `p-year`, and `p-price`. The goal of Entity Matching in this scenario is to decide, for every pair $(e_1, e_2) \in E_1 \times E_2$, if they are referring to the same real-world product.

What makes this non-trivial is that even a genuinely matching pair may disagree on attribute values. For instance, one platform may list a product as *"Sony WH-1000XM5"* while another lists it as *"Sony Noise Cancelling Headphones XM5 Series"*. They are referring to the same product but with different representations. It is therefore insufficient to rely solely on attribute-level comparison to determine a match. To attain maximum accuracy in entity matching, it falls upon a human expert to manually inspect each pair and make a verdict. However, with thousands of listings on each platform, the number of pairs in $E_1 \times E_2$ grows quadratically - making manual inspection labor intensive and expensive. This motivates the need for a more systematic and efficient approach to Entity Matching.

1.2 The Entity Resolution (ER) Pipeline

To address the Entity Matching Problem, we transform it into a classification problem. This is done by passing datasets through an ER Pipeline consisting of three key steps: **Blocking**, **Comparison**, and **Matching**. The first two steps serve as preprocessing stages that progressively reduce and structure the problem, while the third step is what Entity Matching entails and is the primary focus of the paper. We will go over the first two steps in brief before diving into greater detail on the third.

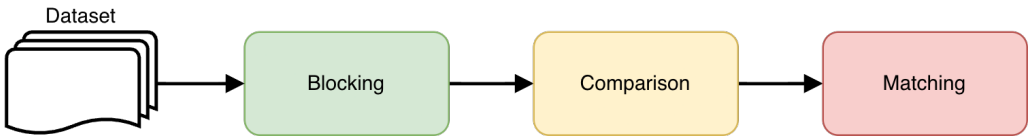

Figure 1: The ER Pipeline.

1.2.1 Blocking

As mentioned earlier in 1.1, the two entity sets E_1 and E_2 can grow quadratically, making it computationally infeasible to compare every pair. The goal of blocking is to narrow the search space down to a smaller candidate subset $S \subseteq E_1 \times E_2$, retaining only pairs that are plausible matches based on an application-dependent heuristic.

Going back to the Shopee-Lazada example, one may restrict S to only those pairs (e_1, e_2) where `p-year` agrees across both listings. Note that blocking is optional — if skipped, $S = E_1 \times E_2$.

Remark 1.1 (Application-Dependent Heuristic)**.** The criterion used for blocking is **application-dependent**; there is no universal rule for what constitutes an "obvious non-match".

For example:

- **E-commerce:** A mismatched `p-year` is a reliable signal to discard a pair.
- **Hospital records:** Mismatched blood type might serve that role.
- **Legal documents:** Neither attribute would reliably apply.

The heuristic is therefore dictated by which attributes are reliable and meaningful in the specific domain.

A notable example of a more general, domain-agnostic blocking heuristic is **Locality Sensitive Hashing (LSH)**. It groups similar pairs into candidate buckets based on hash collisions, this was covered in Chapter 3 of this course.

1.2.2 Comparison

For each remaining pair $(e_1, e_2) \in S$, the comparison step quantifies their similarity across d attributes using a set of similarity functions $sim_1, sim_2, \ldots, sim_d$. The i-th coordinate of the resulting point p_{e_1,e_2} equals $sim_i(e_1, e_2)$, where a higher value indicates greater similarity on that attribute. The choice of similarity function depends on the attribute type:

- For numerical attributes such as `p-price`, one may use $-|e_1.\texttt{price} - e_2.\texttt{price}|$, where the negation ensures consistency with the convention that higher values reflect greater similarity.
- For text attributes such as `p-name` and `p-description`, functions such as edit distance, Jaccard similarity, or cosine similarity may be employed.

Example 1.1. Consider an entity pair $(e_1, e_2) \in S$. e_1 is a Shopee listing and e_2 is a Lazada listing for what may be the same product. Suppose each entity has the attributes `p-name`, `p-description`, `p-year`, and `p-price`. We pass each attribute pair into a pre-defined similarity function as follows:

- `p-name`: a text attribute — use Jaccard similarity. E.g., *"Sony WH-1000XM5"* vs *"Sony Noise Cancelling XM5"* $\rightarrow sim_1(e_1, e_2) = 0.8$
- `p-description`: a long text attribute — use cosine similarity. $\rightarrow sim_2(e_1, e_2) = 0.65$
- `p-year`: a numerical attribute — use exact match, returning 1 if equal and 0 otherwise. $\rightarrow sim_3(e_1, e_2) = 1$
- `p-price`: a numerical attribute — use $-|e_1.\texttt{price} - e_2.\texttt{price}|$. E.g., \$350 vs \$345 $\rightarrow sim_4(e_1, e_2) = -5$

The resulting point representing this pair in $\mathbb{R}^4$ is:

$$p_{e_1,e_2} = (0.8,\ 0.65,\ 1,\ -5)$$

This will be done for every pair in S

This process yields a d-dimensional set of points $P = \{p_{e_1,e_2} \mid (e_1, e_2) \in S\}$ which is handed over to the Matching step.

1.2.3 Matching

The entity matching objective now becomes a binary classification task of assigning each point $p_{e_1,e_2} \in P$ to its correct ground truth label, which is $+1$ for a matching pair and -1 otherwise. While the ultimate approach to this would be with human judgement, this paper introduces **Monotone Classification** as a way to automate the process while ensuring a high degree of labelling accuracy.

1.3 Monotone Classification

Instead of relying on human judgement, we want to find a classifier to correctly label matching and non-matching entities.

The input is the set P of n points in $\mathbb{R}^d$ for $d \geq 1$ that we obtained from the Comparison step. Each point $p \in P$ holds a label from $\{-1, 1\}$ which is represented as $label(p)$.

We first define a classifier as a function:

$$h : \mathbb{R}^d \to \{-1, +1\}$$

where $h(p) = label(p)$ is a correct classification made by h and will be a misclassification otherwise.

The question is, what would make a good classifier in this context?

1.3.1 The Monotonicity Constraint

To answer this, we first introduce the notion of **dominance**. We say that a point $p \in \mathbb{R}^d$ *dominates* another point $q \in \mathbb{R}^d$, written $p \succ q$, if:

$$p[i] \geq q[i] \quad \text{for all } i \in [d]$$

That is, p scores at least as high as q on every attribute. In the context of entity matching, this means that the pair represented by p is at least as similar as the pair represented by q on every feature.

This supports the **Monotonicity Constraint**. A classifier h is said to be **monotone** if:

$$p \succ q \implies h(p) \geq h(q)$$

Intuitively, if a pair p is at least as similar as q on every attribute, it would be contradictory for h to label p as a non-match (-1) while labeling q as a match $(+1)$. A more similar pair should receive at least as positive a label. We denote the set of all monotone classifiers as $\mathbb{H}_{mon}$. So for entity matching, the key criteria of our classifier is that it must be **monotone**.

1.3.2 The Optimal Monotone Error

Given a classifier h, its **error** on P is defined as the number of points it misclassifies:

$$err_P(h) = \sum_{p \in P} \mathbf{1}_{h(p) \neq label(p)}$$

That is, the total count of points in P whose assigned label differs from their ground truth. A classifier $h \in \mathbb{H}_{mon}$ is said to be **optimal** on P if it achieves the smallest possible error among all monotone classifiers. We define this minimum achievable error as the **Optimal Monotone Error**:

$$k^* = \min_{h \in \mathbb{H}_{mon}} err_P(h)$$

Note that k^* need not be zero — in practice, the labels in P may not perfectly obey monotonicity due to noise in the data, meaning even the best monotone classifier will misclassify some points.

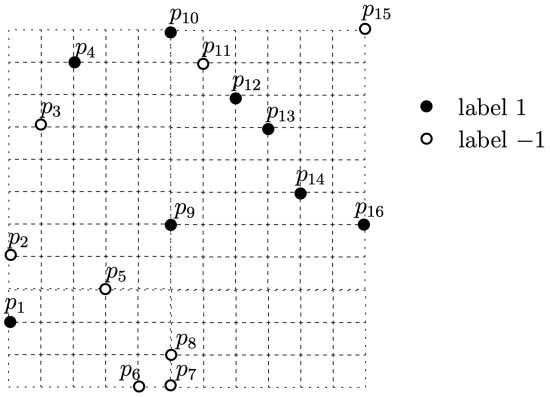

Figure 2: Labelling of input set P by monotone classifier h

Example 1.2. Consider the 2D point set P illustrated in Figure 2 taken from Professor Tao's original paper [2]. Each point $p_{e_1,e_2} \in P$ represents an entity pair with $d = 2$ similarity features. Black points carry label $+1$ (match) and white points carry label -1 (non-match).

The monotone classifier h that maps all black points to $+1$ except p_1, and all white points to -1 except p_{11} and p_{15}, achieves $err_P(h) = 3$. No other monotone classifier does better on P, and hence $k^* = 3$. Note that $k^* > 0$ — this is the non-realizable case, as no monotone classifier can perfectly separate all points in P.

We now want an algorithm $\mathcal{A}$ to find a classifier from $\mathbb{H}_{mon}$ with low error on P. Note that in the beginning, the labels on all points in P are hidden. In order for $\mathcal{A}$ to gain better understanding in forming an optimal classifier, it is allowed to query an *oracle* — a human expert in practice – to reveal the ground truth label of a chosen point $p \in P$. This action is defined as a *probe*. To assess the efficiency of the $\mathcal{A}$, we can measure its cost by the number of *probes* it makes.

To find an optimal monotone classifer on P, one can simply have $\mathcal{A}$ to probe all of P with a cost of n. On the other hand, not probing at all would incur zero cost but the classifier may perform poorly on P. This presents us with the challenge of minimizing the number of probes while still producing a classifier with sufficiently low error.

Formally, given a tolerance parameter $\epsilon \geq 0$, the goal is to determine the minimum number of probes required to guarantee a classifier with error at most $(1+\epsilon) \cdot k^*$. This is defined formally below.

Problem 1.1 (Monotone Classification with Relative Precision Guarantees)**.** Given a real value $\epsilon \geq 0$, find a monotone classifier $h \in \mathbb{H}_{mon}$ whose error on P is at most $(1+\epsilon) \cdot k^*$, while minimizing the number of probes made by $\mathcal{A}$.

Note that when $\epsilon = 0$, the algorithm must find an exactly optimal classifier, which as we will see requires $\Omega(n)$ probes in the worst case — essentially probing all of P. This justifies the introduction of $\epsilon > 0$, where even a small tolerance dramatically reduces the probing cost. The

remainder of this paper studies how efficiently this problem can be solved across the full range of ϵ.

1.4 Active Classification (Supplementary)

To motivate Active Classification, we first revisit a familiar example of **passive classification** — the **Perceptron Algorithm**, covered in SC2300 Machine Learning. Recall that the Perceptron is a mistake-driven algorithm that iterates through all training examples $S_n = \{(x^{(t)}, y^{(t)}), t = 1, \ldots, n\}$, where both the points *and* their labels are fully visible from the start. It updates its parameter vector θ whenever a misclassification occurs, converging to a linear classifier that correctly separates all training examples — provided they are linearly separable. This is the **realizable case**.

Remark 1.2 (Realizable Case)**.** A classification problem is **realizable** if there exists a classifier $h \in \mathbb{H}_{mon}$ that correctly classifies all points in P with zero error, i.e., $k^* = 0$. This is analogous to linear separability in the Perceptron setting.

When data is not linearly separable, the Perceptron fails to converge — motivating the use of **hinge loss** to tolerate a margin of error. The same challenge arises here: in practice, the labels in P rarely obey monotonicity perfectly, meaning no monotone classifier can achieve zero error. This is the **non-realizable case**.

Remark 1.3 (Non-Realizable Case)**.** A classification problem is **non-realizable** if no classifier $h \in \mathbb{H}_{mon}$ can achieve zero error on P, i.e., $k^* > 0$. This arises when the labels in P contain noise — points that violate monotonicity regardless of which classifier is chosen. This is analogous to the non-linearly separable case in SC2300.

Remark 1.4. Just as hinge loss introduces a tolerance for misclassified points in the non-separable case, this paper introduces $(1 + \epsilon)k^*$ as a multiplicative tolerance for the non-realizable case — same philosophy, different context.

In the context of Entity Matching, the non-realizable case is the norm. Furthermore, unlike the Perceptron, labels are **not** visible from the start — they are hidden and can only be revealed through probing. This is where **Active Classification** comes in. Devise a learning strategy where the algorithm selectively chooses which points to probe rather than requiring all labels upfront, gathering just enough evidence to construct a classifier with error at most $(1 + \epsilon)k^*$ while minimizing the number of probes. In other words, Active Classification takes into account cost apart from just accuracy.

The contrast with passive classification is summarized below:

	Passive (Perceptron)	**Active (This Paper)**
Labels	All visible from start	Hidden, revealed via probes
Setting	Realizable ($k^* = 0$)	Non-realizable ($k^* > 0$)
Cost metric	Number of iterations	Number of probes
Goal	Zero training error	$(1 + \epsilon)k^*$ error

Table 1: Passive vs. Active Classification

A key limitation of existing active learning approaches is that they require prior knowledge of k^* — the optimal monotone error — before the algorithm even begins. This is an unrealistic

assumption in practice, since k^* can only be determined after all labels are revealed. This paper proposes algorithms that require no such prior knowledge, working instead by strategically probing points to implicitly converge towards a $(1+\epsilon)$-approximate solution — which we examine in the sections that follow.

2 RPE Algorithm

2.1 The Algorithm

2.2 Why It Works

2.3 Guarantees and Limitations

3 Relative-Comparison Coresets (RCC)

3.1 Why RPE Isn't Enough

Recall that RPE achieves an expected error of at most $2k^*$, and this guarantee is tight. In the entity-matching context a factor-2 margin may be unacceptable for a production pipeline.

The goal of this section is to achieve an approximation ratio of $(1+\varepsilon)$ for any user-specified $\varepsilon > 0$: find a function $F : \mathbb{H}_{\text{mon}} \to \mathbb{R}$ such that

$$F(h) \le F(h') \implies \operatorname{err}_P(h) \le (1+\varepsilon)\operatorname{err}_P(h').$$

Since $F(\hat{h}) \le F(h^*)$ for any optimal h^*, this gives $\operatorname{err}_P(\hat{h}) \le (1+\varepsilon)k^*$.

Remark 3.1 (Why estimating $\operatorname{err}_P(h)$ directly fails)**.** A natural idea is to set $F(h) = \operatorname{err}_P(h)$ estimated by probing. But even for h^{pos} (the all-+1 classifier), estimating its error to within a constant relative factor requires finding the single -1 element (or confirming none exists), which demands $\Omega(n)$ probes in expectation.

3.2 The Unknown-Δ Technique

We sidestep the $\Omega(n)$ barrier by constructing F satisfying a weaker condition. Instead of estimating $\operatorname{err}_P(h)$ exactly, we find an F where all estimates are shifted by the same unknown constant Δ:

$$\operatorname{err}_P(h)\big(1-\tfrac{\varepsilon}{4}\big) + \Delta \;\le\; F(h) \;\le\; \operatorname{err}_P(h)\big(1+\tfrac{\varepsilon}{4}\big) + \Delta \qquad \forall\, h \in \mathbb{H}_{\text{mon}}, \tag{1}$$

Proposition 3.1. Any F satisfying (1) has the relative ε-comparison property.

Proof. Suppose $F(h) \le F(h')$. Using the left inequality of (1) for h and the right for h', then cancelling Δ:

$$\operatorname{err}_P(h)\big(1-\tfrac{\varepsilon}{4}\big) \;\le\; F(h) - \Delta \;\le\; F(h') - \Delta \;\le\; \operatorname{err}_P(h')\big(1+\tfrac{\varepsilon}{4}\big)\,.$$

Rearranging:

$$\operatorname{err}_P(h) \;\le\; \frac{1+\varepsilon/4}{1-\varepsilon/4}\operatorname{err}_P(h') \;\le\; (1+\varepsilon)\operatorname{err}_P(h'),$$

where the last step uses $\frac{1+\varepsilon/4}{1-\varepsilon/4} \le 1+\varepsilon$ for all $\varepsilon \in (0,1]$. □

The key insight: Δ cancels whenever we *compare* two classifiers. We never need to know its value — only its existence.

3.2.1 Running Example

We will trace a small 1D instance through the entire construction. Let $P = \{1,2,3,4,5,6,7,8\}$ with labels

$$\underbrace{-1,-1,-1,-1}_{\text{points 1–4}},\ \underbrace{+1,+1,+1,+1}_{\text{points 5–8}}.$$

Every monotone 1D classifier has the form $h^\tau(p) = +1$ iff $p > \tau$. The optimal classifier is $h_{4.5}$, with $k^* = 0$. We set $\varepsilon = 1/2$.

3.3 The Coreset Technique

We realise F via a **relative-comparison coreset**: a weighted subset $Z \subseteq P$ whose weighted error w-err$_Z(h)$ satisfies (1) for all h.

Definition 3.1 (Relative-Comparison Coreset)**.** A **relative-comparison coreset** of P is a subset $Z \subseteq P$ where every $z \in Z$ has its label revealed and carries a positive weight weight(z), such that the *weighted error*

$$\text{w-err}_Z(h) \;:=\; \sum_{z \in Z : h(z) \neq \text{label}(z)} \text{weight}(z)$$

satisfies (1) for every $h \in \mathbb{H}_{\text{mon}}$ — i.e., w-err$_Z$ serves as the desired function F.

The novelty of the unknown-Δ approach is that classical coresets approximate err$_P(h)$ itself (costing $\Omega(n)$ probes), whereas here Δ never needs to be computed.

3.4 Building the Coreset in 1D: The Recursive Framework

The 1D construction works by solving the following problem at every level of recursion. The coreset is built by the algorithm BuildCoreset, which solves Problem 3.1 recursively.

Problem 3.1 (1D Function Approximation)**.** Let P be a multiset of 1D labeled points (as in Problem 1.1 with $d = 1$), and $\varepsilon \in (0, 1]$. Find a function $F : \mathbb{H}_{\text{mon}} \to \mathbb{R}$ such that every $h \in \mathbb{H}_{\text{mon}}$ satisfies simultaneously:

$$|F(h) - \text{err}_P(h)| \;\le\; \varepsilon|P|/64, \tag{2}$$

$$\text{err}_P(h)(1 - \tfrac{\varepsilon}{4}) + \Delta \;\le\; F(h) \;\le\; \text{err}_P(h)(1 + \tfrac{\varepsilon}{4}) + \Delta, \tag{3}$$

for some (possibly unknown) Δ with $|\Delta| \le \varepsilon|P|/64$.

3.4.1 Estimating Error via Sampling (Functions G_1, G_2, F_α)

At each recursion level on a set $Q \subseteq P$, we draw a uniform sample S of size $O\Big(\frac{1}{\varepsilon^2} \log \frac{|Q|\,\ell}{\delta}\Big)$ from Q, where $\ell = O(\log n)$ is the number of recursion levels. Define

$$G(h) \;:=\; \frac{|Q|}{|S|} \cdot \text{err}_S(h).$$

By a standard concentration bound, G satisfies condition (2) for Q simultaneously for all $h \in \mathbb{H}_{\text{mon}}$, with high probability. Three such estimates are used per level:

- G_1, sampled from the full P, is used to *locate the split point*. Does not enter the coreset.
- G_2, sampled from P_{rest}, approximates err$_{P_{\text{rest}}}(h)$ and contributes its sampled points to Z.
- F_α, sampled from P_α, approximates err$_{P_\alpha}(h)$ and contributes its sampled points to Z.

Together, the target function for Problem 3.1,

$$F(h) = G_2(h) + F_\alpha(h) + F_{\text{mid}}(h)$$

where F_{mid} comes from recursion on P_{mid} satisfies both conditions of Problem 3.1 for the full P.

As there are ℓ levels, the overall cost is $O\Big(\frac{\ell}{\epsilon^2} \log \frac{n}{\delta}\Big) = O\Big(\frac{\log n}{\epsilon^2} \cdot \log \frac{n}{\delta}\Big)$, which lets us sovle Problem 3.1 with a probability of at least $1 - \delta$.

3.4.2 Example (continued)

- Suppose at the top level we draw $S_1 = \{2, 5, 7\}$ from P, with labels $-1, +1, +1$.
- For $h_{4.5}$ (optimal): $\mathrm{err}_{S_1}(h_{4.5}) = 0$, so $G_1(h_{4.5}) = 0$. True error is also 0; condition (2) holds trivially ($|0 - 0| = 0 \leq 0.0625$).
- For $h_{2.5}$: $\mathrm{err}_{S_1}(h_{2.5}) = 1$ (point 5 misclassified), so $G_1(h_{2.5}) = 8/3 \approx 2.67$. True error is 2; the gap is 0.67, well within the tolerance $\varepsilon|P|/64 = 0.0625 \cdot 8 = 0.5$... close to the boundary, illustrating why the sample size matters.

3.4.3 Splitting P into Three Parts

The purpose of G_1 — satisfying condition (2) for the full P — is to identify where the error function transitions through the value $|P|/4$. Concretely, define:

$$\alpha = \text{smallest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\tfrac{1}{4} - \tfrac{\varepsilon}{64}\right),$$
$$\beta = \text{largest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\tfrac{1}{4} - \tfrac{\varepsilon}{64}\right).$$

Because G_1 satisfies (2), any h^τ with $G_1(h^\tau)$ below this threshold has $\mathrm{err}_P(h^\tau) < |P|/4$. This splits P into:

$$P_\alpha = \{p \in P : p = \alpha\}, \quad P_{\text{mid}} = \{p \in P : \alpha < p \leq \beta\}, \quad P_{\text{rest}} = P \setminus (P_\alpha \cup P_{\text{mid}}).$$

Crucially, classifiers h^τ with $\tau \notin [\alpha, \beta)$ have $\mathrm{err}_P(h^\tau) \geq |P|/4$, so condition (3) for those classifiers follows from (2) alone (with $\Delta = 0$). The recursive step only needs to handle $\tau \in [\alpha, \beta)$, where errors are small and the unknown Δ is needed.

Proposition 3.2 (Size of P_{mid})**.** $|P_{\text{mid}}| < |P|/2$.

Proof. If $\alpha = \beta =$ null, the claim is trivial. Otherwise, P_{mid} cannot contain $\geq |P|/4$ positive-label points (else $G_1(h_\beta) \geq |P|(1/4 - \varepsilon/64)$, contradicting the definition of β). By a symmetric argument, it cannot contain $\geq |P|/4$ negative-label points either. Hence $|P_{\text{mid}}| < |P|/2$. □

Proposition 3.2 guarantees the recursion terminates in $O(\log n)$ levels.

3.4.4 Example (continued)

With $|P| = 8$ and $\varepsilon = 1/2$, the threshold is $8(1/4 - 1/128) \approx 1.94$. Suppose G_1 evaluates to roughly:

Classifier h^τ	True $\mathrm{err}_P(h^\tau)$	$G_1(h^\tau)$ (approx.)
$h_{-\infty}$ (all +1)	4	4.0
h_1	3	3.2
h_2	2	2.1
h_3	1	1.2
h_4	0	0.1
h_5	1	0.8
h_6	2	2.2
h_7	3	3.1
h_8 (all −1)	4	4.0

G_1 drops below 1.94 at $\tau = 3$ and $\tau = 4$. So $\alpha = 3$, $\beta = 4$, giving:

$$P_\alpha = \{3\}, \quad P_{\text{mid}} = \{4\}, \quad P_{\text{rest}} = \{1, 2, 5, 6, 7, 8\}.$$

Note $|P_{\text{mid}}| = 1 < 4 = |P|/2$, consistent with Proposition 3.2.

3.4.5 Assembling the Coreset Z

The coreset is assembled level by level:

- A sample S_2 drawn from P_{rest} enters Z with weight $|P_{\text{rest}}|/|S_2|$.
- A sample S_α drawn from P_α enters Z with weight $|P_\alpha|/|S_\alpha|$.
- The recursion on P_{mid} returns a sub-coreset Z_{mid}, which is also included in Z.

Setting $F(h) = G_2(h) + F_\alpha(h) + F_{\text{mid}}(h)$ and $\Delta = \Delta_\alpha + \Delta_{\text{mid}} + (G_2(h_\beta) - \text{err}_{P_{\text{rest}}}(h_\beta))$, one can verify (see Section 3 of the paper) that F satisfies both conditions (2) and (3) of Problem 3.1 for the full P. Since $\text{w-err}_Z(h) = F(h)$, the coreset Z realises the desired function F.

Theorem 3.1 (Theorem 6 in the paper)**.** Let n and w be the size and width of the input P, respectively. In

$$O\left(\frac{w}{\varepsilon^2}\log\frac{n}{w}\cdot\log\frac{n}{\delta}\right)$$

probes, we can obtain with probability at least $1-\delta$ a relative-comparison coreset Z of P with

$$|Z| = O\left(\frac{w}{\varepsilon^2}\log\frac{n}{w}\cdot\log\frac{n}{\delta}\right).$$

3.4.6 Example (continued)

At the top level, S_2 is drawn from $P_{\text{rest}} = \{1,2,5,6,7,8\}$ (size 6) and S_α from $P_\alpha = \{3\}$ (size 1). The recursion on $P_{\text{mid}} = \{4\}$ (size 1) trivially probes the single element. With weights assigned proportionally to the set sizes, w-err_Z correctly scales the sampled errors back to represent all of P.

3.5 Extension to Higher Dimensions

For $d > 1$, the algorithm first computes a **chain decomposition** $C_1, \ldots, C_w$ of P using Dilworth's Theorem — the same decomposition used in the cost analysis of RPE (Section 2). Since each chain C_i is ordered, it can be treated as a 1D instance.

Applying the 1D coreset construction to each C_i independently yields sub-coresets $Z_1, \ldots, Z_w$. The final coreset is $Z = \bigcup_{i=1}^{w} Z_i$. For every $h \in \mathbb{H}_{\text{mon}}$:

$$\text{err}_P(h) = \sum_{i=1}^{w}\text{err}_{C_i}(h), \qquad \text{w-err}_Z(h) = \sum_{i=1}^{w}\text{w-err}_{Z_i}(h).$$

Since each Z_i satisfies a per-chain relative bound with unknown shift Δ_i, summing gives the global bound with $\Delta = \sum_i \Delta_i$.

3.6 Achieving the $(1+\varepsilon)$ Guarantee

Given the coreset Z, finding the best monotone classifier requires *no further probing*: simply compute

$$\hat{h} = \arg\min_{h\in\mathbb{H}_{\text{mon}}} \text{w-err}_Z(h),$$

which can be done in time polynomial in $|Z|$ and d.

Corollary 3.1 (Corollary 7 in the paper)**.** For Problem 1 with any $\varepsilon \in (0, 1]$, there exists an algorithm that finds, with high probability, a monotone classifier with error at most $(1+\varepsilon)k^*$ using

$$O\left(\frac{w}{\varepsilon^2}\log\frac{n}{w}\cdot\log n\right)$$

probes, where $n = |P|$, w is the width of P, and k^* is the optimal monotone error.

The argument is direct: for any $h \in \mathbb{H}_{\text{mon}}$, the coreset bound gives

$$\text{err}_P(\hat{h})(1-\tfrac{\varepsilon}{4}) \le \text{w-err}_Z(\hat{h}) - \Delta \le \text{w-err}_Z(h^*) - \Delta \le \text{err}_P(h^*)(1+\tfrac{\varepsilon}{4})\,,$$

so setting $h = h^*$ (the true optimal) and rearranging yields $\text{err}_P(\hat{h}) \le (1+\varepsilon)k^*$.

3.6.1 Example (conclusion)

We compute $\text{w-err}_Z(h^\tau)$ for each threshold τ. The minimiser is $h_{4.5}$ (equivalently h_4), with weighted error ≈ 0. Since $k^* = 0$, the $(1+\varepsilon)k^* = 0$ bound is met exactly.

4 Full Algorithmic Description

Algorithm 1 BuildCoreset(P)

$Z \leftarrow \emptyset$
if $|P| = 1$ **then**
  Probe the (only) element $p \in P$
  $Z \leftarrow \{p\}$ with weight$(p) = 1$
else
  Probe sample S_1 of size $O\left(\frac{1}{\varepsilon^2}\log\left(\frac{|P|\ell}{\delta}\right)\right)$ from P
  {Defines $G_1(h) = (|P|/|S_1|)\cdot \text{err}_{S_1}(h)$; use G_1 to find α, β and split P into $P_\alpha, P_{\text{mid}}, P_{\text{rest}}$}
  if $P_{\text{rest}} \neq \emptyset$ **then**
    Probe sample S_2 of size $O\left(\frac{1}{\varepsilon^2}\log\left(\frac{|P|\ell}{\delta}\right)\right)$ from P_{rest}
    {Defines $G_2(h) = (|P_{\text{rest}}|/|S_2|)\cdot \text{err}_{S_2}(h)$}
    Add S_2 to Z with weight$(p) = |P_{\text{rest}}|/|S_2|$ for each $p \in S_2$
  end if
  if $P_\alpha \neq \emptyset$ **then**
    Probe sample S_α of size $O\left(\frac{1}{\varepsilon^2}\log\left(\frac{|P|\ell}{\delta}\right)\right)$ from P_α
    {Defines $F_\alpha(h) = (|P_\alpha|/|S_\alpha|)\cdot \text{err}_{S_\alpha}(h)$}
    Add S_α to Z with weight$(p) = |P_\alpha|/|S_\alpha|$ for each $p \in S_\alpha$
  end if
  $Z_{\text{mid}} \leftarrow$ BuildCoreset(P_{mid})
  Add Z_{mid} to Z
end if
return Z

5 Proofs and Lower Bounds

5.1 Lower Bound Results

5.2 Near-Optimality of the Algorithms

5.3 Mathematical Machinery

Summary

References

[1] Y. Tao, "Monotone Classification," *PODS'18*.

[2] Y. Tao, "Monotone Classification (Extended)," *PODS'21*.